

Resolución Parte Teórica

1) Sistemas Operativos - Generalidades

a) ¿Quién es el encargado de administrar el tiempo de la CPU y cuál es su función principal dentro del sistema?

En FreeRTOS, la gestión del tiempo de CPU es responsabilidad del planificador (scheduler), el cual implementa algoritmos de planificación basados en prioridades. Su función principal es determinar, en función del estado del sistema y de las prioridades asignadas, cuál de las tareas listas para ejecutarse debe tomar el control del procesador. El scheduler permite la conmutación de contexto entre tareas, de modo que una tarea de mayor prioridad puede interrumpir la ejecución de una tarea de menor prioridad. De esta manera, se garantiza una asignación eficiente de los recursos del sistema en tiempo real.

b) Explique a que se denomina “Contexto de Ejecución”. ¿Cuáles son las variables intervinientes?

El contexto de ejecución de una tarea se define como el conjunto completo de información que representa su estado en un instante determinado. Este incluye registros de CPU, punteros de pila, contador de programa y cualquier otro dato necesario para que el sistema operativo pueda reanudar la ejecución de la tarea de manera precisa tras una conmutación. Durante cada cambio de contexto, el sistema debe almacenar el contexto de la tarea saliente y restaurar el de la tarea entrante, asegurando así que cada tarea retome su ejecución exactamente desde el punto en el que fue interrumpida.

El contexto de ejecución suele incluir:

Registros del CPU: En el contexto de la arquitectura ARM Cortex-M, el conjunto de registros que conforman el estado de una tarea incluye tanto los registros de propósito general (R0–R12) como registros clave del sistema, tales como el contador de programa (PC), el puntero de pila (SP), el registro de enlace (LR) y el registro de estado del programa (xPSR). Estos registros capturan el estado exacto de la CPU en el momento en que una tarea es interrumpida, y deben ser almacenados adecuadamente para permitir una restauración precisa durante la reanudación de la tarea.

Estado de la tarea: Cada tarea en FreeRTOS mantiene un descriptor que incluye su estado actual dentro del sistema operativo. Este estado puede ser lista, bloqueada, suspendida o en ejecución, entre otros. El scheduler utiliza esta información para determinar qué tareas son elegibles para ejecución. Sólo las tareas en estado lista participan en la selección, y entre ellas se elige aquella con la mayor prioridad. Este mecanismo es fundamental para garantizar una planificación determinista y eficiente, especialmente en sistemas embebidos basados en ARM Cortex-M.

Variables de estado: Se refiere a la información adicional asociada a cada tarea que permite al sistema operativo realizar su gestión de manera eficiente. Estas variables suelen incluir la prioridad asignada,

identificadores únicos, punteros a estructuras internas, y otros metadatos relevantes. Esta información es utilizada por el planificador y otros componentes del sistema operativo para controlar el ciclo de vida de la tarea, tomar decisiones de planificación, manejar eventos de sincronización y garantizar un comportamiento coherente del sistema.

c) ¿A qué se denomina Sistema Operativo de Tiempo Real? ¿Por qué usar un Sistema Operativo de Tiempo Real?

Sistema Operativo de Tiempo Real (RTOS): Un sistema operativo de tiempo real es un tipo de software diseñado para gestionar la ejecución de tareas en sistemas donde el cumplimiento de restricciones temporales es fundamental. Su principal característica es la capacidad de proporcionar respuestas deterministas ante eventos, garantizando que las tareas críticas se ejecuten dentro de límites de tiempo definidos. Esta previsibilidad en la temporización lo hace ideal para aplicaciones embebidas, de control industrial, aeroespaciales, entre otras, donde el fallo en el tiempo de respuesta puede comprometer la funcionalidad o la seguridad del sistema.

Se emplea en aplicaciones donde el cumplimiento estricto de los tiempos de ejecución es crítico, ya que, como se mencionó anteriormente, un RTOS está diseñado para garantizar respuestas dentro de plazos predefinidos. Un ejemplo representativo es el sistema de activación de un airbag: ante la detección de una colisión, el sistema debe ejecutar la tarea correspondiente en una ventana de tiempo muy reducida, ignorando o postergando cualquier otra tarea que estuviera en ejecución. Cualquier demora en este contexto podría comprometer la seguridad del sistema y de sus usuarios.

2) Kernel de FreeRTOS

a) Explique y desarrolle las características de una “Tarea” en FreeRTOS. Proponga un ejemplo de una tarea manteniendo la sintaxis correcta.

Tareas en FreeRTOS:

Una tarea (task) representa una unidad básica de ejecución gestionada por el sistema operativo. Permiten implementar concurrencia en sistemas embebidos, posibilitando la ejecución de múltiples operaciones de forma pseudo-simultánea. En el contexto de sistemas basados en microcontroladores como el STM32 —frecuentemente utilizado en entornos académicos—, las tareas se utilizan para estructurar la lógica del programa en bloques funcionales independientes y cooperativos.

Cada tarea se implementa como una función en lenguaje C que contiene un bucle infinito, ya que en un sistema de tiempo real no se espera que una tarea finalice su ejecución. A diferencia de las funciones tradicionales, las tareas en FreeRTOS están diseñadas para ejecutarse bajo control del scheduler, y deben ser eficientes en el uso de recursos debido a las limitaciones propias de los sistemas embebidos.

Características principales de una tarea en FreeRTOS:

- **Prioridad:** Cada tarea tiene una prioridad asociada que determina el orden en el que se ejecuta. Las tareas de mayor prioridad pueden interrumpir a las de menor prioridad si están en ejecución. Se ejecutan de forma concurrente bajo control del planificador.
- **Función de Tarea:** Cada tarea está implementada mediante una función que contiene el código que se ejecutará mientras la tarea esté activa. Esta función debe ser de tipo void y no debe tener parámetros asociados.
- **Pila (Stack):** Cada tarea tiene su propia pila que se utiliza para almacenar las variables locales, registros y otros datos necesarios para la ejecución de la tarea. El tamaño de la pila debe ser especificado al crear la tarea. Interactúan con otras tareas mediante mecanismos de sincronización y comunicación (colas, semáforos, etc.).
- **Estado de Tarea:** Una tarea puede estar en uno de varios estados, entre ellos:
 - **Listo (Ready):** La tarea está lista para ejecutarse y está esperando que el planificador la elija para su ejecución.
 - **En ejecución (Running):** La tarea está actualmente siendo ejecutada por el CPU.
 - **Bloqueado (Blocked):** La tarea está esperando un evento o condición para continuar, como la recepción de datos en una cola o la liberación de un semáforo, por ejemplo.
 - **Suspendida (Suspended):** La tarea ha sido suspendida manualmente y no se ejecutará hasta que se reanude explícitamente.
- **Interrupciones y Contexto:** Cuando se produce una interrupción o una conmutación de contexto, FreeRTOS guarda el contexto de la tarea actual (como los registros y el puntero de pila) y lo restaura cuando la tarea se reanuda.
- **Sincronización y Comunicación:** FreeRTOS proporciona mecanismos para la sincronización y comunicación entre tareas, como colas y semáforos, lo que permite a las tareas colaborar y coordinarse eficazmente.

Ejemplo de tarea:

```
#include "FreeRTOS.h"

#include "task.h"

// Función de tarea

void vTaskFunction(void *pvParameters)

{
    while(1)
    {
```

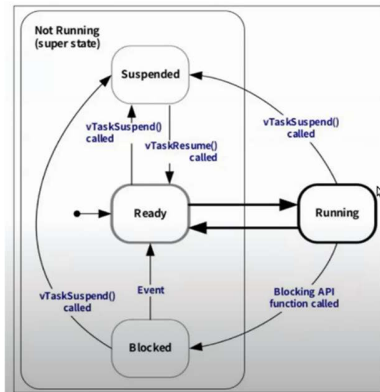
```

        // Código de la tarea
    }
}

// Crear
una tarea
void
createTask()
{
    xTaskCreate(
        vTaskFunction, // Función que ejecutará la tarea
        "TaskName",    // Nombre de la tarea
        1000,          // Tamaño de la pila (en palabras)
        NULL,          // Parámetro de entrada para la tarea
        1,             // Prioridad de la tarea
        NULL            // Handle de la tarea (opcional)
    );
}

```

b) Explique y desarrolle los “Estados de una Tarea” y sus transiciones. Indique las funciones que permiten las transiciones.



1. Ready (Listo)

Descripción:

Una tarea en estado Ready está preparada para ejecutarse y es considerada por el planificador en el próximo ciclo de planificación. Aunque aún no está en ejecución, cumple con todas las condiciones necesarias para ser seleccionada por el scheduler en función de su prioridad.

Transición:

Una tarea entra al estado Ready en distintos escenarios: al ser creada y añadida al sistema, tras despertar de un estado Blocked (por ejemplo, luego de que se complete una espera en una cola, semáforo o temporizador), o al ser reanudada desde un estado Suspended. Si hay varias tareas en estado Ready, el scheduler elige aquella con mayor prioridad para ejecutarse.

2. Running (En Ejecución)

Descripción:

Una tarea en estado Running es aquella que está siendo ejecutada activamente por el procesador. En sistemas con un solo núcleo (CPU único), únicamente una tarea puede estar en este estado en un momento dado. Este estado representa el punto en el que la tarea tiene el control del CPU.

Transición:

Una tarea entra al estado Running cuando el scheduler la selecciona como la tarea de mayor prioridad entre las que están en estado Ready. Puede salir de este estado si ocurre alguna de las siguientes condiciones:

- Una tarea de mayor prioridad se vuelve elegible y provoca una conmutación de contexto.
- La tarea se bloquea voluntariamente (por ejemplo, al esperar en una cola o semáforo).
- La tarea se suspende explícitamente.
- El sistema realiza un cambio de tarea por razones de planificación (como en el caso del time slicing, si está habilitado para tareas de igual prioridad).

3. Blocked (Bloqueado)

Descripción:

Una tarea entra en estado *Blocked* cuando se encuentra a la espera de que ocurra un evento o se cumpla una condición que le permita continuar con su ejecución. Durante este estado, la tarea no consume tiempo de CPU, lo que permite que el procesador se asigne a otras tareas listas. Las condiciones típicas que provocan este estado incluyen la espera de datos en una cola, la disponibilidad de un semáforo, o el cumplimiento de un retardo temporizado.

- **Transición:**

Una tarea pasa al estado *Blocked* como resultado de una llamada a funciones del sistema que implican espera, como `xQueueReceive()` cuando la cola está vacía, o `vTaskDelay()` para generar un retardo. Permanecerá bloqueada hasta que el evento o condición esperada se cumpla (por ejemplo, se recibe un mensaje en la cola, o expira el temporizador de retardo), momento en el cual transita nuevamente al estado *Ready*, quedando disponible para ser seleccionada por el scheduler.

- **4. Suspended (Suspendida)**

Descripción:

Una tarea en estado *Suspended* ha sido suspendida explícitamente y no es considerada por el scheduler para su ejecución, independientemente de su prioridad o del estado del sistema. Mientras permanece en este estado, la tarea no consume tiempo de CPU ni recursos de planificación, aunque su contexto se mantiene almacenado en memoria para su posterior reactivación.

- **Transición:**

Una tarea entra al estado *Suspended* mediante una llamada a la función `vTaskSuspend()`. Para que la tarea vuelva a estar activa, debe ser reanudada manualmente mediante la función `vTaskResume()` (o `xTaskResumeFromISR()` si se realiza desde una interrupción). Una vez reanudada, la tarea transita al estado *Ready* si cumple con las condiciones para ser ejecutada.

`vTaskResume()` o `xTaskResumeFromISR()` si se encuentra el llamado dentro de una interrupción.

Por su parte, un listado de las funciones que permiten transicionar entre tareas:

- `xTaskCreate()`: Crea una nueva tarea y la añade al sistema de gestión de tareas de FreeRTOS. La tarea pasa al estado de "Ready" y espera ser seleccionada para su ejecución.
- `vTaskSuspend()`: Suspende una tarea, moviéndola al estado "Suspended". La tarea no se ejecutará hasta que se reanude explícitamente.
- `vTaskResume()`: Reanuda una tarea que estaba en el estado "Suspended", moviéndola al estado "Ready".
- `xTaskResumeFromISR()`: Similar a `vTaskResume()`, pero se utiliza dentro de una rutina de servicio de interrupción (ISR por sus siglas en inglés). Permite reanudar una tarea desde una ISR, moviéndola al estado "Ready".
- `xQueueReceive()`: Bloquea la tarea hasta que se recibe un mensaje en la cola o hasta que se cumple un tiempo de espera. Si la cola está vacía, la tarea pasa al estado "Blocked".

- `xQueueSend()`: Envía un mensaje a una cola. Si la cola está llena, la tarea puede bloquearse hasta que haya espacio disponible en la cola, moviéndola al estado "Blocked".
- `vTaskDelay()`: Bloquea la tarea durante un período de tiempo específico. La tarea pasa al estado "Blocked" durante el retraso y regresa al estado "Ready" una vez que el tiempo ha pasado.

c) ¿A que nos referimos con el concepto de “tareas de procesamiento continuo”? ¿y a que nos referimos con el concepto de “tareas manejadas por eventos”?

Una tarea de procesamiento continuo en un sistema operativo de tiempo real (RTOS), como FreeRTOS, es aquella que ejecuta su lógica de manera ininterrumpida, sin requerir la ocurrencia de eventos externos para iniciar su funcionamiento. Este tipo de tareas está diseñada para operar dentro de un bucle infinito —comúnmente implementado como `while(1)`— y se mantiene activa durante todo el tiempo en que el sistema está operativo.

Estas tareas suelen encargarse de operaciones de monitoreo, control o procesamiento que deben realizarse de forma constante, como el escaneo de sensores, generación de señales, o gestión de periféricos. Dado que consumen continuamente tiempo de CPU, es fundamental asignarles una prioridad adecuada y, en muchos casos, incluir retardos o mecanismos de espera voluntaria (`vTaskDelay()`) para evitar la saturación del procesador.

Por otro lado, una tarea basada en eventos (event-driven task) se ejecuta únicamente en respuesta a la ocurrencia de eventos específicos. A diferencia de las tareas de procesamiento continuo, estas no permanecen activas constantemente ni operan en intervalos regulares, sino que permanecen bloqueadas hasta que se cumpla una condición previamente definida, como la llegada de un mensaje a una cola, la activación de un semáforo, o la señalización desde una interrupción.

Este tipo de tareas es especialmente eficiente en sistemas donde el ahorro de recursos y el tiempo de CPU son críticos, ya que solo se despiertan cuando realmente es necesario realizar una acción. Su diseño favorece un comportamiento reactivo, ideal para la gestión de entradas del usuario, comunicaciones, o eventos generados por periféricos.

d) ¿Cuáles son los tipos de eventos por los que una tarea puede estar esperando al entrar en el estado bloqueado? Explicar.

Eventos que pueden desbloquear una tarea

En un sistema operativo de tiempo real, una tarea puede entrar en el estado Blocked cuando debe esperar a que se cumpla una condición determinada antes de continuar su ejecución. Estos eventos actúan como disparadores que, una vez ocurren, provocan que la tarea pase al estado Ready, quedando disponible para ser seleccionada por el scheduler.

Los principales tipos de eventos que pueden provocar el desbloqueo de una tarea son:

- Recepción de datos en una cola: La tarea espera hasta que un mensaje esté disponible en una cola mediante funciones como `xQueueReceive()`.
- Disponibilidad de un semáforo o mutex: La tarea queda bloqueada hasta que un recurso compartido esté libre y pueda adquirir el semáforo o mutex correspondiente (`xSemaphoreTake()`).
- Finalización de un retardo temporal: Utilizando funciones como `vTaskDelay()` o `vTaskDelayUntil()`, la tarea se bloquea durante un intervalo de tiempo determinado y se despierta automáticamente al expirar el tiempo.
- Interrupciones (ISRs – Interrupt Service Routines)
Descripción:
Si bien una tarea no se bloquea esperando directamente una interrupción, las ISRs pueden generar eventos que provocan el desbloqueo de tareas previamente suspendidas. Es decir, la ISR actúa como un disparador indirecto: al ocurrir una interrupción, esta ejecuta código que puede liberar un recurso, enviar una señal o actualizar un estado que una tarea estaba esperando.
- Un caso típico es el de una ISR que libera un semáforo, lo cual permite que una tarea bloqueada por la espera de ese semáforo transite al estado Ready y eventualmente sea ejecutada.
- Función asociada:
- `xSemaphoreGiveFromISR()` – Utilizada dentro de la ISR para liberar un semáforo de forma segura.
- Ejemplo:
Una interrupción generada por un periférico (como una UART al recibir un byte) puede ejecutar una ISR que, al terminar de procesar el evento, libera un semáforo. Una tarea bloqueada esperando dicho semáforo puede entonces reanudarse para continuar con el procesamiento de los datos recibidos.
- Notificación de tarea: A través de mecanismos como `xTaskNotify()` o `xTaskNotifyGive()`, otras tareas o rutinas de interrupción pueden enviar señales que desbloquean una tarea que esté esperando una notificación.
- Eventos de sincronización personalizados: En sistemas más complejos, se pueden definir condiciones específicas que combinen múltiples eventos para controlar el desbloqueo de una tarea.

e) ¿Cuál es la función de la IDLE TASK? ¿Puede el procesador no estar ejecutando ninguna tarea en algún momento? Explicar.

Idle Task (Tarea de Inactividad)

En FreeRTOS, la Idle Task es una tarea especial creada automáticamente por el sistema cuando se

invoca la función `vTaskStartScheduler()`. Esta tarea tiene la prioridad más baja posible y siempre se encuentra en estado Ready, pero solo se ejecuta cuando no existen otras tareas listas con mayor prioridad.

La función principal de la Idle Task es ocupar el procesador durante los períodos en los que no hay trabajo pendiente, evitando que el sistema quede inactivo sin control. Además, puede servir como punto de extensión para tareas de mantenimiento del sistema o de gestión de bajo consumo energético, ya que FreeRTOS permite al usuario definir un Idle Hook (función de usuario que se ejecuta dentro del contexto de la tarea Idle).

Características clave:

- Se ejecuta únicamente cuando no hay tareas listas de mayor prioridad.
- Tiene la prioridad más baja del sistema.
- No debe bloquearse (es decir, no debe llamar a funciones que puedan ponerla en estado Blocked).
- Puede usarse para implementar funciones como eliminación de tareas o entrada en modos de bajo consumo.

Esta tarea existe porque, en un sistema operativo de tiempo real como FreeRTOS, el procesador no puede quedar completamente inactivo sin ejecutar ninguna tarea. Debido al modelo de planificación utilizado por el scheduler, siempre debe haber al menos una tarea disponible para ejecución. La Idle Task cumple esta función, garantizando que el CPU tenga una tarea que ejecutar en todo momento, incluso cuando no haya tareas de aplicación listas. De este modo, se mantiene la estabilidad del sistema y se proporciona un punto de control para gestionar los períodos de inactividad.

3) Desarrolle como es el almacenamiento de datos en el recurso “Cola” proporcionado por el Kernel.

Una cola en FreeRTOS es un mecanismo fundamental de comunicación y sincronización entre tareas, que permite el intercambio de datos de manera estructurada y segura. Está implementada como una estructura de datos FIFO (First In, First Out), lo que significa que los elementos se almacenan en el orden en que son enviados (en queue) y se recuperan en el mismo orden (de queue).

Las colas permiten que tareas o interrupciones puedan enviar datos a otras tareas sin necesidad de compartir variables globales, evitando así condiciones de carrera. Además, pueden ser utilizadas como mecanismo de sincronización: una tarea puede bloquearse esperando datos en una cola, reanudando su ejecución automáticamente cuando estos estén disponibles.

4) Explique utilizando un ejemplo con código el porqué es común que el recurso Cola posea múltiples tareas escritoras y una sola tarea lectora. Explique el proceso de bloqueo por escritura y bloqueo por lectura en el recurso Cola.

En FreeRTOS, las colas son recursos compartidos que no pertenecen a ninguna tarea en particular. Cualquier tarea que tenga una referencia válida a la cola puede enviar o recibir datos, lo que permite establecer canales de comunicación entre múltiples tareas. Es común que varias tareas productoras escriban datos en una misma cola, mientras una tarea consumidora se encarga de leer y procesar esos datos.

Este patrón es especialmente útil en sistemas donde múltiples fuentes generan información que debe ser tratada de manera centralizada. Por ejemplo, en un sistema embebido con varios sensores (como velocidad, temperatura y presión), cada sensor puede tener su propia tarea encargada de adquirir datos y enviarlos a una cola común. Una única tarea de procesamiento puede entonces recibir los datos desde la cola y realizar el análisis o transmisión correspondiente.

Ejemplo de código básico:

```
QueueHandle_t xColaSensores;
```

```
void tareaSensorVelocidad(void *pvParameters) {  
    int velocidad = 90; // Valor simulado  
    xQueueSend(xColaSensores, &velocidad, portMAX_DELAY);  
    vTaskDelay(pdMS_TO_TICKS(200));  
}
```

```
void tareaSensorTemperatura(void *pvParameters) {  
    int temperatura = 25; // Valor simulado  
    xQueueSend(xColaSensores, &temperatura, portMAX_DELAY);  
    vTaskDelay(pdMS_TO_TICKS(200));  
}
```

```
void tareaProcesadora(void *pvParameters) {  
    int datoRecibido;  
    while (1) {  
        if (xQueueReceive(xColaSensores, &datoRecibido, portMAX_DELAY) == pdPASS) {  
            // Procesar el dato recibido  
        }  
    }
```

```

    }
}

void main_app() {
    xColaSensores = xQueueCreate(10, sizeof(int));

    xTaskCreate(tareaSensorVelocidad, "SensorVel", 128, NULL, 2, NULL);
    xTaskCreate(tareaSensorTemperatura, "SensorTemp", 128, NULL, 2, NULL);
    xTaskCreate(tareaProcesadora, "Procesador", 128, NULL, 1, NULL);

    vTaskStartScheduler();
}

```

Cuando una tarea intenta leer datos desde una cola en FreeRTOS, puede especificar un tiempo de espera (timeout). Si en ese momento la cola está vacía, la tarea entra en estado Blocked durante el período definido, esperando a que otra tarea (o una ISR) escriba un elemento en la cola. Tan pronto como se coloca un dato en la cola, la tarea que estaba bloqueada pasa automáticamente al estado Ready y queda disponible para ser ejecutada por el scheduler.

En situaciones donde múltiples tareas están esperando datos de la misma cola, se aplica la política de planificación del sistema. Si las tareas tienen diferentes prioridades, se desbloquea primero la de mayor prioridad. En cambio, si tienen la misma prioridad, se desbloquea aquella que haya estado esperando por más tiempo, siguiendo una política de tipo FIFO entre iguales.

En todos los casos, si no llega ningún dato antes de que se agote el tiempo de espera especificado, la función de lectura retorna y la tarea continúa su ejecución, normalmente con un código de error o manejo de timeout.

Al igual que en las operaciones de lectura, FreeRTOS permite que las tareas que escriben en una cola especifiquen un tiempo de bloqueo. Si la cola se encuentra llena al momento del intento de escritura, la tarea puede bloquearse durante un período determinado, a la espera de que se libere espacio (es decir, que otro elemento sea leído y eliminado de la cola).

Durante ese tiempo, la tarea entra en estado Blocked. Si se libera espacio antes de que expire el tiempo de espera, la tarea pasa al estado Ready y puede retomar la operación de escritura. En caso contrario, si se agota el tiempo sin que se libere espacio, la función retorna con un valor que indica que no fue posible insertar el dato.

Cuando múltiples tareas están bloqueadas esperando para escribir en la misma cola, el scheduler aplica la política de planificación establecida. Se da prioridad a la tarea de mayor prioridad, o bien, si varias tareas comparten el mismo nivel de prioridad, se desbloquea primero aquella que haya estado esperando por más tiempo.

5) A la hora de enviar datos compuestos: explique cómo se ' lleva a cabo dicho proceso y por qué no genera conflictos con la definición de la macro del recurso. ¿Cuál sería la implementación si los datos fueran “grandes”, lo que ' ocasionaría un problema con la memoria RAM?

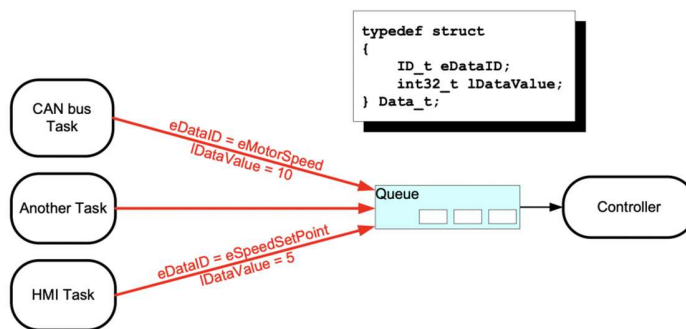


Fig. 1. Recibir datos de múltiples fuentes

Cuando se requiere enviar datos compuestos —es decir, conjuntos de múltiples variables relacionadas— a través de una cola en FreeRTOS, es necesario encapsular esa información en una estructura de datos definida por el usuario. Para ello, se utiliza una estructura (struct) junto con un typedef, creando un nuevo tipo de dato que agrupa todos los campos necesarios (por ejemplo: timestamp, identificador, valores medidos, etc.).

Este enfoque no genera conflictos con la definición de la cola mediante la macro `xQueueCreate()`, ya que simplemente se utiliza el nuevo tipo definido con typedef como argumento en la creación de la cola. De este modo, cada elemento encolado tendrá la forma y tamaño de la estructura personalizada, permitiendo un intercambio de información más organizado y coherente entre tareas.

Cuando los datos que se desean enviar a través de una cola en FreeRTOS son de gran tamaño —como estructuras complejas con múltiples campos—, es recomendable no enviar directamente la estructura completa, sino pasar un puntero a la misma. Esto reduce el uso de memoria RAM y mejora la

eficiencia del sistema, ya que evita la copia de bloques de memoria grandes en cada operación de envío y recepción.

Sin embargo, al utilizar punteros en lugar de copias directas de datos, es necesario tener en cuenta una serie de consideraciones importantes para garantizar la integridad del sistema y evitar errores difíciles de depurar:

Consideraciones al enviar punteros por cola:

1. Validez del puntero:
Asegurarse de que el puntero que se envía apunte a una región de memoria válida durante todo el tiempo que esté en uso por otra tarea. No se debe enviar el puntero a una variable local que pueda salir del alcance (por ejemplo, variables locales de funciones).
2. Evitar condiciones de carrera:
Si varias tareas acceden o modifican el contenido apuntado, se debe utilizar algún mecanismo de sincronización (como semáforos o mutexes) para evitar inconsistencias o corrupción de datos.
3. Responsabilidad de la memoria:
Es fundamental definir claramente quién es responsable de reservar y liberar la memoria apuntada. Si se utiliza `malloc()` para crear el dato, debe existir un control adecuado para su posterior liberación, evitando memory leaks o double free.
4. Persistencia de los datos:
La memoria apuntada debe seguir siendo válida después de que el puntero haya sido enviado por la cola y hasta que la tarea receptora haya terminado de procesarla.

6) Explicar el funcionamiento del esquema de la fig 1:

En este esquema, múltiples tareas del sistema generan información relevante para el control de un motor —como velocidad actual o valores de referencia (setpoint)— y la transmiten mediante una cola. Para ello, se utiliza un tipo de dato definido por el usuario a través de una estructura (struct) que agrupa ambos valores, facilitando el envío conjunto de datos relacionados.

La cola es leída exclusivamente por una tarea controladora, que se encarga de interpretar los datos recibidos y ajustar los parámetros del motor en consecuencia. Esta tarea actúa como único punto de acceso al hardware del motor, garantizando un control centralizado y evitando conflictos por acceso concurrente.

Es importante destacar que, si bien varias tareas pueden enviar información de control, estas no interactúan directamente con el motor. Además, la prioridad o relevancia de los datos puede variar

entre tareas emisoras, por lo que es responsabilidad de la tarea controladora evaluar cada mensaje recibido y decidir cómo actuar en base a su contenido o contexto.

7) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Semáforos Mutex”.

```
#include "FreeRTOS.h"

#include "task.h"

#include "semphr.h"

#include <stdio.h>

// Declarar el handle del semáforo mutex
SemaphoreHandle_t xMutex;

// Función de la primera tarea
void vTask1(void *pvParameters)
{
    while(1)
    {
        // Intentar tomar el mutex
        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
        {
            // Acceder al recurso compartido
            printf("Tarea 1 está accediendo al recurso.\n");

            // Simular trabajo con el recurso
            vTaskDelay(pdMS_TO_TICKS(1000));

            // Liberar el mutex
            xSemaphoreGive(xMutex);
```

```

    }

    // Esperar antes de intentar acceder nuevamente
    vTaskDelay(pdMS_TO_TICKS(1000));

}

}

// Función de la segunda tarea void
vTask2(void *pvParameters)
{
    while(1)
    {
        // Intentar tomar el mutex
        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
        {
            // Acceder al recurso compartido

            printf("Tarea 2 está accediendo al recurso.\n");

            // Simular trabajo con el recurso
            vTaskDelay(pdMS_TO_TICKS(1000));

            // Liberar el mutex
            xSemaphoreGive(xMutex);
        }

        // Esperar antes de intentar acceder nuevamente
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

void setup()
{
    // Crear el semáforo mutex

```

```

xMutex = xSemaphoreCreateMutex();

if (xMutex != NULL)
{
    // Crear las tareas

    xTaskCreate(vTask1, "Task1", 1000, NULL, 1, NULL);

    xTaskCreate(vTask2, "Task2", 1000, NULL, 1, NULL);

}
else
{
    // Manejar el error si el semáforo no se creó
    printf("Error al crear el semáforo mutex.\n");
}
}

```

8) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Inversión de Prioridad”. ‘ Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

```

#include "FreeRTOS.h"

#include "task.h"

#include "queue.h"

#include <stdio.h>

// Handles para las tareas

TaskHandle_t xTaskHighPriorityHandle = NULL;
TaskHandle_t xTaskLowPriorityHandle = NULL;

// Handle para la cola

QueueHandle_t xQueue;

```



```

// Función de la tarea de alta prioridad
void vHighPriorityTask(void *pvParameters)
{
    char message = 'A';
    while(1)
    {
        // Intentar enviar un mensaje a la cola
        if (xQueueSend(xQueue, &message, portMAX_DELAY) == pdTRUE)
        {
            printf("Tarea de alta prioridad ha enviado un mensaje.\n");
        }
        else
        {
            printf("Tarea de alta prioridad falló al enviar un mensaje.\n");
        }

        // Esperar antes de intentar enviar el siguiente mensaje
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// Función de la tarea de baja prioridad
void vLowPriorityTask(void *pvParameters)
{
    char receivedMessage;
    while(1)
    {
        // Intentar recibir un mensaje de la cola
        if (xQueueReceive(xQueue, &receivedMessage, portMAX_DELAY) == pdTRUE)
        {

```

```

    printf("Tarea de baja prioridad ha recibido el mensaje: %c\n", receivedMessage);

    // Simular una operación prolongada
    vTaskDelay(pdMS_TO_TICKS(5000));
}
}

// Función de la tarea de prioridad intermedia void
vMediumPriorityTask(void *pvParameters)
{
    while(1)
    {
        printf("Tarea de prioridad intermedia está ejecutándose.\n");
        vTaskDelay(pdMS_TO_TICKS(1000)); // Esperar antes de la siguiente ejecución
    }
}

void setup()
{
    // Crear la cola
    xQueue = xQueueCreate(10, sizeof(char));

    if (xQueue != NULL)
    {
        // Crear tareas
        xTaskCreate(vHighPriorityTask, "HighPriorityTask", 1024, NULL, 3, &xTaskHighPriorityHandle);
        xTaskCreate(vLowPriorityTask, "LowPriorityTask", 1024, NULL, 1, &xTaskLowPriorityHandle);
        xTaskCreate(vMediumPriorityTask, "MediumPriorityTask", 1024, NULL, 2, NULL);
    }
}

```

```

else

{
    // Manejar el error si la cola no se creó
    printf("Error al crear la cola.\n");
}

}

int main(void)

{
    setup();

    vTaskStartScheduler(); // Iniciar el planificador de FreeRTOS

    while(1); // No debería llegar aquí
}

```

El código es simple, la cola se utiliza para enviar mensajes entre tareas. La tarea de alta prioridad envía mensajes a la cola, y la tarea de baja prioridad los recibe. La tarea de alta prioridad (3) está esperando que la tarea de baja prioridad (1) libere el recurso (la cola). La tarea de prioridad intermedia (2) puede ejecutarse mientras la tarea de alta prioridad está bloqueada esperando que la tarea de baja prioridad complete su operación.

En este caso, propusimos como la tarea de baja prioridad está ocupada con una operación prolongada (simulada en la línea `vTaskDelay (5000)`), la tarea de alta prioridad queda bloqueada esperando que la cola esté disponible. Durante este tiempo, la tarea de prioridad intermedia (2) puede ejecutarse, lo que demuestra la inversión de prioridades.

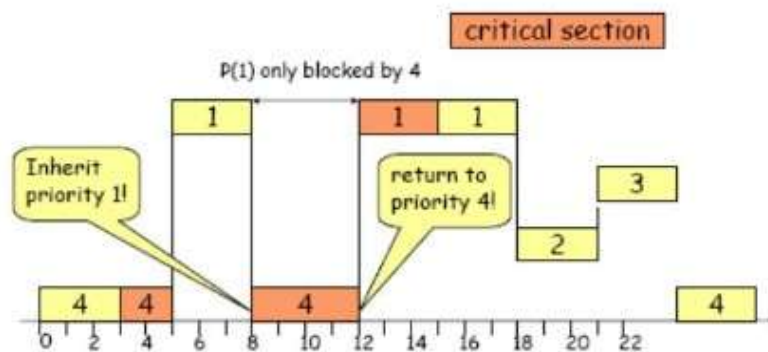
En este ejemplo, se implementa una cola para enviar mensajes entre tareas con diferentes niveles de prioridad. La tarea de alta prioridad (prioridad 3) se encarga de enviar mensajes a la cola, mientras que la tarea de baja prioridad (prioridad 1) los recibe. Sin embargo, si la cola está llena y la tarea de baja prioridad tarda en liberar espacio, la tarea de alta prioridad se bloquea, quedando a la espera de que la operación de recepción se complete.

Durante este período de bloqueo, la tarea de prioridad intermedia (prioridad 2), que no está involucrada directamente en el intercambio de datos con la cola, puede ejecutarse sin restricciones. Esto genera un escenario clásico de inversión de prioridades, en el cual una tarea de alta prioridad se ve indirectamente bloqueada por una de baja prioridad, mientras una tarea intermedia continúa ejecutándose, aunque no debería retrasar al sistema bajo una política estricta de prioridades.

En el ejemplo propuesto, esta condición se simula al introducir un retardo artificial (`vTaskDelay (5000)`) en la tarea de baja prioridad. Durante ese lapso, la cola no es liberada, y, por ende, la tarea de alta prioridad no puede continuar. La ejecución de la tarea intermedia durante este tiempo resalta la

importancia de aplicar mecanismos como herencia de prioridad (priority inheritance) cuando se requiere garantizar determinismo en sistemas críticos.

9) Explicar el concepto de “Herencia de Prioridad” utilizando diagramas gráficos que permitan comprenderlo.



El diagrama ilustra un escenario en el que una tarea de baja prioridad (prioridad 4) está bloqueando a una tarea de alta prioridad (prioridad 1), debido a que posee un recurso que la tarea crítica necesita para continuar su ejecución. En esta situación, se activa el mecanismo de herencia de prioridad (priority inheritance), mediante el cual la tarea de baja prioridad "hereda" temporalmente la prioridad de la tarea de mayor prioridad que está esperando.

Esta elevación de prioridad se mantiene únicamente durante el tiempo necesario para liberar el recurso compartido. Una vez que la tarea de baja prioridad libera dicho recurso, su prioridad vuelve automáticamente a su valor original. Este mecanismo es fundamental para evitar la inversión de prioridades, una condición que podría causar retrasos inaceptables en la ejecución de tareas críticas.

La herencia de prioridad es especialmente importante en sistemas operativos de tiempo real como FreeRTOS, ya que ayuda a preservar la predictibilidad del sistema y asegura que las tareas de alta prioridad sean despachadas en tiempo oportuno, manteniendo así la eficiencia y el cumplimiento de restricciones temporales estrictas.

10) Desarrollar un programa ejemplo que permita comprender el concepto de “Punto Muerto”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

El punto muerto (deadlock) en sistemas multitarea ocurre cuando dos o más tareas quedan bloqueadas indefinidamente al esperar recursos que están siendo retenidos entre sí. Esta condición se da cuando existe una dependencia circular: cada tarea está esperando que otra libere un recurso que, a su vez,

necesita para continuar. Como resultado, ninguna de las tareas puede avanzar, y el sistema queda parcialmente o totalmente detenido.

Un ejemplo clásico de deadlock se presenta con el uso de colas o semáforos. Supongamos que la Tarea A tiene acceso exclusivo a una cola y está esperando un recurso que posee la Tarea B, mientras que la Tarea B, a su vez, intenta acceder a la misma cola ocupada por la Tarea A. Si ambas permanecen bloqueadas esperando al otro, se genera una situación de bloqueo circular que el sistema no puede resolver por sí solo.

```
#include "FreeRTOS.h"

#include "task.h"

#include "queue.h"

#include <stdio.h>


// Handles para las tareas
TaskHandle_t xTask1Handle = NULL;
TaskHandle_t xTask2Handle = NULL;


// Handle para la cola
QueueHandle_t xQueue;

// Buffer para los mensajes char msgBuffer;

// Función de la primera tarea
void vTask1(void *pvParameters)
{
    while(1)
    {
        // Enviar un mensaje a la cola
        if (xQueueSend(xQueue, &msgBuffer, portMAX_DELAY) == pdTRUE)
        {
            printf("Tarea 1 ha enviado un mensaje.\n");
        }

        // Intentar recibir un mensaje de la cola (lo que causa deadlock)
```

```

    if (xQueueReceive(xQueue, &msgBuffer, portMAX_DELAY) == pdTRUE)
    {
        printf("Tarea 1 ha recibido un mensaje.\n");
    }

    // Esperar antes de cotinuar
    vTaskDelay(pdMS_TO_TICKS(1000));
}
}

```

```

// Función de la segunda tarea void
vTask2(void *pvParameters)
{
    while(1)
    {
        // Intentar recibir un mensaje de la cola (lo que causa deadlock)
        if (xQueueReceive(xQueue, &msgBuffer, portMAX_DELAY) == pdTRUE)
        {
            printf("Tarea 2 ha recibido un mensaje.\n");
        }

        // Enviar un mensaje a la cola
        if (xQueueSend(xQueue, &msgBuffer, portMAX_DELAY) == pdTRUE)
        {
            printf("Tarea 2 ha enviado un mensaje.\n");
        }
    }
}

```

```

    }

    // Esperar antes de continuar
    vTaskDelay(pdMS_TO_TICKS(1000));

}

}

void setup()

{
    // Crear la cola con espacio para un solo elemento
    xQueue = xQueueCreate(1, sizeof(char));

    if (xQueue != NULL)

    {
        // Crear las tareas

        xTaskCreate(vTask1, "Task1", 1000, NULL, 1, &xTask1Handle);

        xTaskCreate(vTask2, "Task2", 1000, NULL, 1, &xTask2Handle);

    }

    else

    {
        // Manejar el error si la cola no se creó
        printf("Error al crear la cola.\n");

    }

}

int main(void)

{
    setup();

    vTaskStartScheduler(); // Iniciar el planificador de FreeRTOS

    while(1); // No debería llegar aquí

}

```